

Flask Films

Version procédurale REST simple - lecture guidée du code, avec extraits visibles mais non sélectionnables.

Support enseignant - code visible rasterisé

Objectif du support

Comprendre comment une application Flask simple peut séparer routes, logique métier, accès SQL et templates.

Identifier ce que REST change dans la conception des URL et dans le choix des méthodes HTTP.

Comprendre le rôle de chaque partie du code avant de le réécrire soi-même.

Protection appliquée : les pages sont rendues comme images. Le code est visible, mais il n'est pas sélectionnable ni copiable directement.

Ce document sert de guide de lecture pour un projet Flask volontairement simple : pas de classe, pas de Blueprint, pas de SQLAlchemy, pas de repository séparé. Le code est montré pour pouvoir être compris, mais le PDF reste rasterisé pour limiter le copier-coller direct.

Ce que l'étudiant doit retenir

Une route Flask reçoit une requête HTTP et choisit quoi faire selon la méthode.

Le service porte les règles métier et les validations.

Le model contient les requêtes SQL et la gestion transactionnelle.

Ce qui reste à faire par l'étudiant

Le code est visible, mais il doit être retapé, testé et compris.

Les explications indiquent le rôle de chaque fonction et de chaque couche.

L'OCR reste possible, mais l'objectif pédagogique est d'éviter le simple copier-coller.

Consigne possible pour les étudiants

- Reproduire la structure du projet à partir du schéma.
- Écrire les routes REST à partir du tableau des ressources.
- Retaper les extraits utiles en vérifiant que chaque ligne est comprise.
- Tester chaque méthode HTTP avec le navigateur ou avec un outil comme curl/Postman.

Support pédagogique - code non copiable

Structure du projet

Arborescence simplifiée

```
01 fims_procedural_rest/  
02 |─ main.py                # routes Flask + commandes CLI  
03 |─ config.py             # variables de configuration MySQL  
04 |─ schema.sql            # table films  
05 |─ requirements.txt  
06 |─ app/  
07 |   |─ __init__.py        # création de l'application Flask  
08 |   |─ database.py        # connexion + exécution du schéma SQL  
09 |   |─ models/  
10 |       |─ film_model.py  # requêtes SQL CRUD  
11 |   |─ services/  
12 |       |─ film_service.py # validations + règles métier  
13 |   |─ templates/  
14 |       |─ base.html  
15 |       |─ films/  
16 |           |─ list.html  
17 |           |─ form.html  
18 |           |─ detail.html
```

Fichier	Rôle
main.py	Traduit les requêtes HTTP en appels au service.
film_service.py	Valide les données et applique les règles métier.
film_model.py	Exécute les requêtes SQL avec mysql-connector.
templates/	Produit les pages HTML et le JavaScript minimal.
database.py	Ouvre les connexions et exécute schema.sql.

Rappel : ce que REST change

REST pousse à raisonner en ressources plutôt qu'en actions. L'URL désigne une chose ; la méthode HTTP indique l'action générale à effectuer sur cette chose.

Action métier	Méthode	URI	Sens REST
Lister	GET	/films	Lire la collection
Créer	POST	/films	Ajouter une ressource dans la collection
Afficher	GET	/films/<id>	Lire une ressource précise
Modifier	PUT	/films/<id>	Remplacer/mettre à jour cette ressource
Supprimer	DELETE	/films/<id>	Retirer cette ressource

Les anciennes routes `/films/<id>/edit` et `/films/<id>/delete` mélangent l'action dans l'URL. Dans cette version, l'action est portée par PUT ou DELETE.

Point important

- REST ne veut pas dire 'retourner forcément du JSON'. Une application web peut servir du HTML tout en utilisant des URL plus REST.
- Ici, GET renvoie des templates HTML, tandis que PUT et DELETE renvoient du JSON pour être traités par `fetch()`.
- Le projet est donc un compromis pédagogique : REST dans les routes, interface HTML simple côté navigateur.

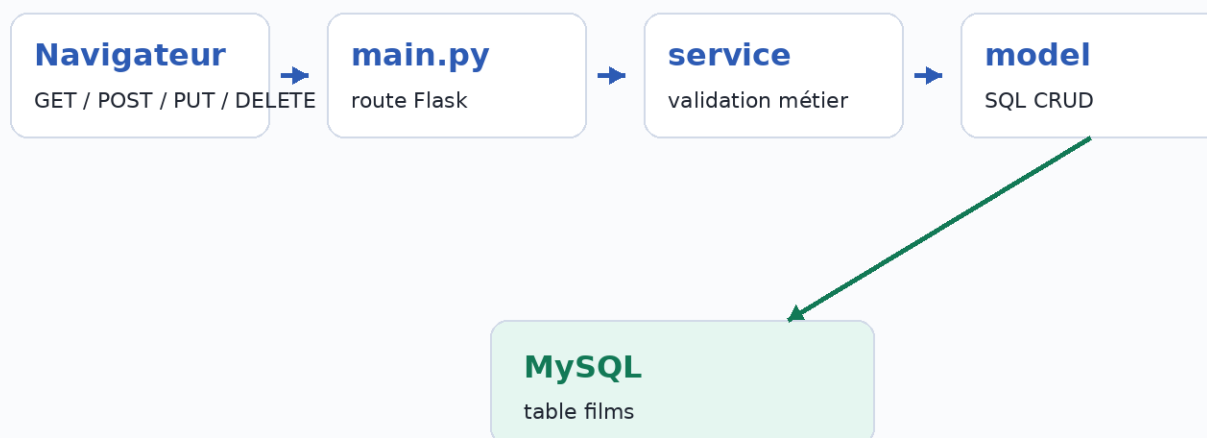
Routes Flask de la version REST

Route	Méthodes	Fonction	Responsabilité
/	GET	home	Rediriger vers la liste des films.
/films	GET, POST	films_collection	Lire ou créer dans la collection.
/films/new	GET	new_film_form	Afficher le formulaire de création.
/films/<id>	GET, PUT, DELETE	film_resource	Lire, modifier ou supprimer un film.

main.py - squelette des routes

```
01 @app.route("/films", methods=["GET", "POST"])
02 def films_collection():
03     # collection de films : liste ou création
04     ...
05
06 @app.route("/films/<int:film_id>", methods=["GET", "PUT", "DELETE"])
07 def film_resource(film_id):
08     # ressource individuelle : lire, modifier ou supprimer
09     ...
```

/films/new n'est pas une action REST de modification. C'est une page de confort pour afficher le formulaire HTML de création ; la création réelle reste POST /films.



Deux types de réponses

- GET /films et GET /films/<id> rendent des pages HTML avec render_template().
- POST /films redirige après création : le navigateur arrive sur la page du nouveau film.

Pourquoi cette séparation ?

La route ne doit pas contenir toutes les règles : elle orchestre. Le service valide. Le model parle à la base. Cela rend le code plus lisible et plus testable, même sans classes.

Extrait guidé - création et liste

```
01 @app.route("/films", methods=["GET", "POST"])
02 def films_collection():
03     if request.method == "GET":
04         films = film_service.list_films()
05         return render_template("films/list.html", films=films)
06
07     try:
08         film_id = film_service.create_film(
09             title=request.form.get("title"),
10             release_year=request.form.get("release_year"),
11             director=request.form.get("director"),
12         )
13         flash("Film créé.", "success")
14         return redirect(url_for("film_resource", film_id=film_id))
15     except ValueError as error:
16         flash(str(error), "error")
17         return render_template("films/form.html", ...)
```

Lecture

- La même URL /films sert à deux choses, selon la méthode HTTP.
- GET lit la collection : c'est une opération sans modification de la base.
- POST crée une nouvelle ressource : les champs arrivent dans request.form.
- En cas de succès, la route redirige vers /films/<id>, ce qui évite de recréer le film si l'utilisateur rafraîchit la page.

Extrait guidé - ressource individuelle

```
01 @app.route("/films/<int:film_id>", methods=["GET", "PUT", "DELETE"])
02 def film_resource(film_id):
03     if request.method == "GET":
04         film = film_service.get_film(film_id)
05         return render_template("films/detail.html", film=film)
06
07     if request.method == "PUT":
08         film_service.update_film(
09             film_id=film_id,
10             title=request.form.get("title"),
11             release_year=request.form.get("release_year"),
12             director=request.form.get("director"),
13         )
14         return jsonify({"success": True, "message": "Film modifié."})
15
16     if request.method == "DELETE":
17         film_service.delete_film(film_id)
18         return jsonify({"success": True, "redirect_url": url_for("films_collection")})
```

Lecture REST

- L'identifiant dans l'URL désigne le film concerné.
- GET lit la ressource et affiche sa page de détail.
- PUT modifie la ressource existante avec les valeurs reçues.
- DELETE supprime la ressource et renvoie au JavaScript l'URL de retour vers la collection.

app/services/film_service.py - modification

```
01 def update_film(film_id, title, release_year, director):
02     get_film(film_id)
03
04     title, year, director = validate_film_data(title, release_year, director)
05
06     updated_rows = film_model.update(
07         film_id=film_id,
08         title=title,
09         release_year=year,
10         director=director,
11     )
12
13     if updated_rows == 0:
14         raise ValueError("Aucun film modifié.")
15
16     return film_model.find_by_id(film_id)
```

Validation	Raison pédagogique
Titre obligatoire	Empêcher l'insertion de données vides.
Réalisateur obligatoire	Montrer qu'une règle métier n'est pas une règle SQL uniquement.
Année entière	Convertir les chaînes du formulaire en données utilisables.
Année plausible	Séparer la cohérence métier de la route Flask.

Le service appelle `get_film()` avant `update/delete` : cela permet d'avoir une erreur claire si l'identifiant n'existe pas.

Service : validate_film_data()

Cette fonction est volontairement isolée dans le service. Elle est appelée à la fois par `create_film()` et par `update_film()`, afin que la création et la modification appliquent exactement les mêmes règles.

app/services/film_service.py - validation centralisée

```
01 def validate_film_data(title, release_year, director):
02     title = (title or "").strip()
03     director = (director or "").strip()
04
05     if not title:
06         raise ValueError("Le nom du film est obligatoire.")
07
08     if len(title) > 150:
09         raise ValueError("Le nom du film ne doit pas dépasser 150 caractères.")
10
11     if not director:
12         raise ValueError("Le réalisateur est obligatoire.")
13
14     if len(director) > 150:
15         raise ValueError("Le réalisateur ne doit pas dépasser 150 caractères.")
16
17     try:
18         year = int(release_year)
19     except (TypeError, ValueError):
20         raise ValueError("L'année de sortie doit être un nombre entier.")
21
22     current_year = date.today().year
23
24     if year < 1888:
25         raise ValueError("L'année de sortie semble trop ancienne.")
26
27     if year > current_year + 5:
28         raise ValueError("L'année de sortie semble trop éloignée dans le futur.")
29
30     return title, year, director
```

À expliquer

- `(title or '')` évite une erreur si le champ est absent ou `None`.
- `strip()` retire les espaces inutiles avant de tester si le champ est vide.
- `int(release_year)` transforme la valeur du formulaire, reçue comme chaîne, en entier.
- `raise ValueError(...)` permet à la route d'afficher ou de renvoyer un message clair.
- La fonction retourne des données nettoyées : `title`, `year`, `director`. Le reste du service utilise donc des valeurs fiables.

app/models/film_model.py - requête SQL paramétrée

```
01 def update(film_id, title, release_year, director):
02     connection = get_connection()
03     cursor = connection.cursor()
04
05     try:
06         cursor.execute(
07             """
08             UPDATE films
09             SET title = %s, release_year = %s, director = %s
10             WHERE id = %s
11             """
12             (title, release_year, director, film_id),
13         )
14         connection.commit()
15         return cursor.rowcount
16     except Exception:
17         connection.rollback()
18         raise
19     finally:
20         cursor.close()
21         connection.close()
```

Points à expliquer

- Les %s ne sont pas des interpolations Python : ce sont des marqueurs transmis à mysql-connector.
- Le tuple de valeurs évite de fabriquer la requête SQL par concaténation de chaînes.
- commit() valide l'écriture ; rollback() annule en cas d'erreur.
- finally ferme toujours le curseur et la connexion, même si une exception survient.

form.html et detail.html - extraits

```
01 <form method="post" action="{{ form_action }}">
02     <input name="title" value="{{ request.form.get('title', '') }}">
03     <input name="release_year" type="number">
04     <input name="director">
05     <button type="submit">Créer avec POST</button>
06 </form>
07
08 <form id="edit-film-form" action="{{ url_for('film_resource', film_id=film.id) }}">
09     <!-- pas de method="put" en HTML classique -->
10     <button type="submit">Enregistrer avec PUT</button>
11 </form>
```

Pourquoi deux comportements ?

Le formulaire de création utilise POST car HTML le gère nativement.

La modification utilise un formulaire visuel, mais le JavaScript intercepte l'envoi pour déclencher une requête PUT.

La suppression n'est pas un formulaire POST vers /delete : c'est une requête DELETE envoyée vers /films/<id>.

À faire vérifier aux étudiants

- Le lien de la liste pointe vers la ressource individuelle : `url_for('film_resource', film_id=film.id)`.
- La page détail contient à la fois l'affichage du film et le formulaire de modification.
- Les messages utilisateur sont affichés soit par `flash()`, soit par JavaScript dans la zone `#message`.

detail.html - fetch()

```
01 document.getElementById("edit-film-form").addEventListener("submit", function (event) {
02     event.preventDefault();
03     const form = event.target;
04     const formData = new FormData(form);
05
06     fetch(form.action, {
07         method: "PUT",
08         body: formData
09     })
10     .then(function (response) { return response.json(); })
11     .then(function (data) {
12         // afficher le message renvoyé par Flask
13     });
14 });
15
16 fetch(event.target.dataset.url, { method: "DELETE" })
17 .then(function (response) { return response.json(); })
18 .then(function (data) { window.location.href = data.redirect_url; });
```

Instruction	Rôle
event.preventDefault()	Empêche l'envoi HTML classique du formulaire.
new FormData(form)	Récupère les champs sans les reconstruire à la main.
fetch(..., method: 'PUT')	Envoie la méthode HTTP manquante en HTML.
response.json()	Lit la réponse produite par jsonify().

code non copiable

database.py et main.py - extraits

```
01 def get_connection():
02     return mysql.connector.connect(
03         host=config.MYSQL_HOST,
04         port=config.MYSQL_PORT,
05         user=config.MYSQL_USER,
06         password=config.MYSQL_PASSWORD,
07         database=config.MYSQL_DATABASE,
08         charset="utf8mb4",
09         collation="utf8mb4_unicode_ci",
10     )
11
12 @app.cli.command("init-db")
13 def init_db():
14     execute_schema("schema.sql")
```

Commande	Effet
flask --app main.py init-db	Crée la table définie dans schema.sql.
flask --app main.py seed-db	Initialise la table puis ajoute quelques films d'exemple si elle est vide.
flask --app main.py run --debug	Lance l'application en mode développement.

La configuration par variables d'environnement permet de changer d'utilisateur, de base ou d'hôte MySQL sans modifier le code source.

Exercice 1 - Cartographier les routes

- Associer chaque action utilisateur à une méthode HTTP et à une URL.
- Expliquer pourquoi /films/<id>/edit et /films/<id>/delete ne sont plus nécessaires.

Exercice 2 - Compléter la route PUT

- Récupérer les trois champs depuis request.form.
- Appeler le service de modification.
- Retourner un JSON de succès ou un JSON d'erreur avec un code 400.

Exercice 3 - Tester sans navigateur

Commandes de test

```
01 curl -X GET http://127.0.0.1:5000/films
02 curl -X POST -d "title=..." -d "release_year=..." -d "director=..." http://127.0.0.1:5000/films
03 curl -X PUT -d "title=..." -d "release_year=..." -d "director=..." http://127.0.0.1:5000/films/1
04 curl -X DELETE http://127.0.0.1:5000/films/1
```

Exercice 4 - Amélioration REST

- Remplacer certains retours JSON par des codes HTTP plus précis : 201 Created, 404 Not Found, 204 No Content.
- Discuter la différence entre PUT et PATCH.
- Proposer une version API pure qui ne rendrait plus de templates HTML.

Idée principale

La version REST ne consiste pas seulement à changer les noms des routes. Elle oblige à séparer la ressource visée (/films ou /films/<id>) de l'opération réalisée (GET, POST, PUT, DELETE).

Concept	À retenir
Collection	/films représente l'ensemble des films.
Ressource	/films/<id> représente un film précis.
Méthode HTTP	Elle porte le verbe : lire, créer, modifier, supprimer.
Service	Il concentre les validations et les règles métier.
Model	Il contient le SQL et la gestion commit/rollback.
JavaScript	Il sert ici uniquement à envoyer PUT et DELETE.

Limite honnête : un PDF image peut toujours être lu manuellement ou OCRisé. La meilleure protection pédagogique reste l'exigence de compréhension : demander aux étudiants d'expliquer, tester et modifier le code qu'ils retapent.